

Laporan Proyek Akhir

SignDoc-Sistem Tanda Tangan Digital untuk Dokumen

Disusun untuk memenuhi proyek akhir pada mata kuliah Kriptografi

Dosen Pengampu: Lia Farhatuaini, S.Kom., M.Cs



Disusu Oleh Kelompok SignSecure:

Ahmad Aly	(2488010069)
Nazwa Hummairah Shiddiqin	(2488010039)
Nisa Naila	(2488010024)

**PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU TARBIYAH DAN KEGURUAN
UIN SIBER SYEKH NURJATI CIREBON
2026 / 1447 H**

A. Pendahuluan

1. Latar Belakang

Di era transformasi digital, dokumen elektronik banyak digunakan dalam bidang bisnis, pendidikan, dan administrasi karena lebih praktis dan mudah diakses. Namun, dokumen digital memiliki kelemahan karena dapat dimodifikasi dengan mudah tanpa terlihat secara langsung. Berbeda dengan dokumen fisik yang menggunakan tanda tangan basah, dokumen digital memerlukan sistem keamanan untuk menjaga keaslian dan integritas data.

Untuk mengatasi masalah tersebut, dikembangkan SignDoc berbasis web yang digunakan untuk penandatanganan dan verifikasi dokumen digital secara aman. Sistem ini menerapkan algoritma RSA untuk proses digital signature dan SHA-256 untuk menjaga integritas dokumen. Selain itu, SignDoc juga dilengkapi dengan fitur QR Code verification, multi-signer, dan ekspor laporan verifikasi PDF untuk meningkatkan keamanan, kecepatan, dan kemudahan dalam proses validasi dokumen digital.

2. Rumusan Masalah

1. Bagaimana membangun sistem tanda tangan digital berbasis web yang aman?
2. Bagaimana memastikan integritas dokumen digital menggunakan teknik kriptografi?
3. Bagaimana menerapkan algoritma RSA dan SHA-256 dalam proses digital signature?
4. Bagaimana menyediakan mekanisme verifikasi dokumen yang mudah digunakan oleh pengguna?
5. Bagaimana mendukung proses penandatanganan dokumen oleh lebih dari satu pihak (multi-signer)?

3. Tujuan

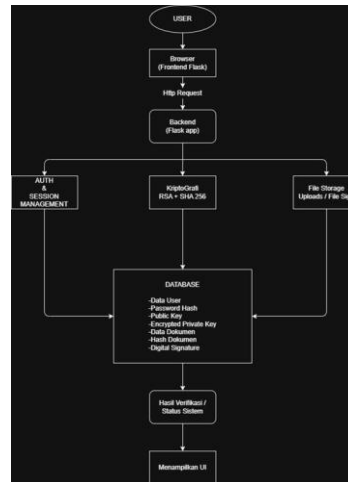
1. Membangun sistem tanda tangan digital berbasis web yang aman dengan mengimplementasikan algoritma RSA untuk proses penandatanganan dan verifikasi dokumen, serta SHA-256 untuk menjaga integritas dokumen.
2. Menyediakan mekanisme verifikasi dokumen yang mudah digunakan melalui QR Code untuk memastikan keaslian dokumen digital.
3. Mendukung proses penandatanganan oleh lebih dari satu pengguna (multi-signer) dalam satu dokumen secara terstruktur.
4. Menghasilkan laporan hasil verifikasi dokumen dalam format PDF untuk keperluan dokumentasi dan pelaporan.

4. Pengguna yang Dituju

1. Mahasiswa dan pihak kampus yang membutuhkan validasi dokumen digital seperti tugas akhir dan dokumen akademik.
2. Perusahaan kecil dan menengah yang membutuhkan penandatanganan dokumen digital yang aman dan efisien
3. Instansi atau bagian administrasi yang sering melakukan pertukaran dokumen digital dan memerlukan jaminan keaslian dokumen.
4. Pengguna umum yang ingin memastikan keaslian serta integritas dokumen digital.

B. Arsitektur Sistem

1. Diagram Arsitektur



2. Komponen Sistem

a. Frontend

Frontend pada SignDoc dibuat menggunakan HTML, CSS, JavaScript, dan template engine Jinja2 dari Flask. Frontend berfungsi untuk proses registrasi, login, upload dokumen, digital signature, verifikasi dokumen, dan melihat riwayat dokumen. Selain itu, frontend juga menampilkan hasil proses dari backend seperti status upload, hasil verifikasi signature, dan pesan error. File frontend utama berada pada folder `templates` dan `static`.

b. Backend

Backend pada SignDoc dibuat menggunakan Flask dengan bahasa pemrograman Python. Backend berfungsi untuk mengelola logika utama sistem dan memproses permintaan pengguna. Proses yang ditangani meliputi autentikasi, upload dokumen, digital signature, dan verifikasi dokumen. Backend menghubungkan sistem dengan database serta modul kriptografi yang digunakan. Selain itu, backend mengatur penyimpanan dokumen dan file signature ke dalam server. Struktur utama backend meliputi `app.py`, folder `routes`, folder `crypto`, dan `database.py`.

c. Database

Database pada SignDoc menggunakan MySQL sebagai media penyimpanan data sistem. Database digunakan untuk menyimpan data pengguna, data dokumen, hash dokumen, public key, private key terenkripsi, serta data digital signature. Selain itu, database juga berfungsi untuk mendukung proses autentikasi pengguna dan verifikasi dokumen.

d. Modul Kriptografi

Modul kriptografi pada aplikasi SignDoc digunakan untuk menjalankan proses keamanan dokumen digital. Sistem menggunakan algoritma RSA 2048-bit untuk proses digital signature dan SHA-256 untuk proses hashing dokumen. Modul ini menangani proses generate keypair, hashing dokumen, pembuatan signature, dan

verifikasi signature. File utama modul kriptografi berada pada folder crypto seperti `keygen.py`, `hashing.py`, `signing.py`, dan `verify.py`.

e. File Storage

File storage pada aplikasi SignDoc digunakan untuk menyimpan dokumen yang diupload pengguna dan file digital signature yang dihasilkan sistem. Dokumen disimpan pada folder uploads, sedangkan file signature disimpan pada folder signatures di dalam server. File tersebut digunakan kembali pada proses penandatanganan dan verifikasi dokumen.

f. QR Code dan PDF Report

Sistem SignDoc juga dilengkapi dengan modul QR Code dan laporan PDF. QR Code digunakan untuk menyediakan akses cepat ke halaman verifikasi dokumen secara publik, sedangkan modul PDF berfungsi untuk menghasilkan laporan hasil verifikasi dokumen yang dapat diunduh oleh pengguna sebagai bukti validasi.

3. Alur Data

a. Registrasi Pengguna

1. Pengguna yang belum memiliki akun melakukan registrasi pada sistem.
2. Sistem melakukan hashing password menggunakan `bcrypt`.
3. Sistem menghasilkan pasangan kunci RSA (public key dan private key).
4. Private key kemudian dienkripsi menggunakan password pengguna.
5. Seluruh data pengguna disimpan ke dalam database.

b. Tanda Tangan Dokumen

1. Pengguna login dan mengupload dokumen yang akan ditandatangani
2. Sistem menghitung hash dokumen menggunakan SHA-256..
3. Sistem mendekripsi private key pengguna.
4. Hash dokumen ditandatangani menggunakan algoritma RSA.
5. Signature hasil penandatanganan disimpan oleh sistem.
6. Sistem menghasilkan QR Code untuk proses verifikasi.

c. Verifikasi Dokumen

1. Pengguna mengunggah dokumen untuk diverifikasi.
2. Sistem menghitung ulang hash dokumen menggunakan SHA-256.
3. Sistem mengambil public key pengguna dari database.
4. Sistem melakukan verifikasi terhadap signature menggunakan RSA.
5. Sistem menampilkan status verifikasi (valid atau tidak valid).

4. Skema Komunikasi

Komunikasi pada SignDoc menggunakan model client-server, di mana client mengirimkan request ke server melalui HTTP/HTTPS. Server yang dibangun menggunakan Flask akan menerima dan memproses setiap permintaan sesuai fitur yang digunakan, seperti login, upload dokumen, penandatanganan, dan verifikasi.

Pada SignDoc, server Flask terhubung dengan database MySQL untuk menyimpan dan mengambil data pengguna, dokumen, serta tanda tangan digital. Proses kriptografi seperti hashing SHA-256 dan penandatanganan RSA dilakukan oleh crypto engine. Sistem juga menggunakan session untuk mengelola status login pengguna, kemudian hasil proses dikirim kembali ke client dalam bentuk response.

C. Desain Kriptografi

1. Algoritma yang Dipilih

a. RSA 2048-bit

Algoritma RSA digunakan sebagai algoritma utama pada proses digital signature dan verifikasi dokumen di aplikasi SignDoc. Sistem menggunakan pasangan public key dan private key yang dibuat secara otomatis saat pengguna melakukan registrasi akun. Private key digunakan untuk proses penandatanganan dokumen, sedangkan public key digunakan untuk proses verifikasi signature. Pada implementasinya, sistem menggunakan RSA 2048-bit karena memiliki tingkat keamanan yang cukup baik dan umum digunakan pada sistem keamanan modern. Proses generate keypair dilakukan pada file `keygen.py`.

b. SHA-256

Algoritma SHA-256 digunakan untuk proses hashing dokumen sebelum proses digital signature dilakukan. Sistem tidak langsung menandatangani isi dokumen, melainkan hash dari dokumen tersebut. Hash yang dihasilkan memiliki panjang tetap dan akan berubah apabila terdapat perubahan sekecil apa pun pada isi dokumen. Pada aplikasi ini, SHA-256 digunakan untuk menjaga integritas dokumen sehingga sistem dapat mendeteksi apakah dokumen telah dimodifikasi setelah proses penandatanganan dilakukan. Proses hashing dokumen diimplementasikan pada file `hashing.py`.

c. Bcrypt

bcrypt digunakan untuk mengamankan password pengguna sebelum disimpan ke database. Password tidak disimpan dalam bentuk plaintext, melainkan diubah menjadi hash menggunakan bcrypt agar lebih aman apabila database mengalami kebocoran data. Pada proses login, sistem akan mencocokkan password yang dimasukkan pengguna dengan hash password yang tersimpan di database. Implementasi bcrypt digunakan pada proses registrasi dan autentikasi pengguna di file `auth.py`.

d. RSA-PSS Padding

Pada proses digital signature, sistem menggunakan RSA-PSS (Probabilistic Signature Scheme) sebagai padding pada algoritma RSA. Penggunaan RSA-PSS bertujuan untuk meningkatkan keamanan signature yang dihasilkan dibandingkan padding lama seperti PKCS1v15. RSA-PSS diterapkan pada proses signing dan verifikasi signature yang terdapat pada file `signing.py` dan `verify.py`. Dengan penggunaan padding ini, signature yang dihasilkan menjadi lebih aman terhadap beberapa jenis serangan kriptografi.

2. Alasan Pemilihan Algoritma

a. RSA-2048

RSA dipilih karena menggunakan mekanisme public key dan private key yang sesuai untuk proses digital signature pada aplikasi SignDoc. Pada sistem ini, private key digunakan untuk proses penandatanganan dokumen, sedangkan public key digunakan untuk proses verifikasi signature dokumen. Selain itu, RSA 2048-bit dipilih karena memiliki tingkat keamanan yang cukup baik dan masih banyak digunakan pada sistem keamanan digital saat ini.

b. SHA-256

SHA-256 dipilih karena mampu menghasilkan hash yang aman dan sulit dimanipulasi. Pada aplikasi SignDoc, algoritma ini digunakan untuk menjaga integritas dokumen sehingga setiap perubahan pada isi dokumen dapat dideteksi oleh sistem, maka hasil verifikasi signature akan berubah dan dokumen dinyatakan tidak valid. Selain itu, SHA-256 memiliki proses yang cukup cepat dan masih banyak digunakan pada berbagai sistem keamanan digital.

c. Bcrypt

bcrypt digunakan untuk proses hashing password pada aplikasi SignDoc. Algoritma ini dipilih karena memiliki tingkat keamanan yang cukup baik untuk melindungi password pengguna sebelum disimpan ke database. Proses hashing pada bcrypt dilakukan secara lebih lambat dibanding hashing biasa sehingga lebih tahan terhadap serangan brute force. Dengan penggunaan bcrypt, password tidak disimpan dalam bentuk asli sehingga keamanan data pengguna dapat lebih terjaga.

d. RSA-PSS Padding

RSA-PSS digunakan sebagai metode padding pada proses digital signature di aplikasi SignDoc. Penggunaan RSA-PSS dipilih karena memiliki tingkat keamanan yang lebih baik dibanding metode padding lama seperti PKCS1v15. Metode ini membantu meningkatkan keamanan signature yang dihasilkan sistem sehingga lebih sulit dipalsukan atau dimanipulasi. Selain itu, RSA-PSS juga masih banyak digunakan pada implementasi digital signature modern karena dinilai lebih aman untuk proses penandatanganan dokumen digital.

3. Alternatif yang Dipertimbangkan dan Analisis Perbandingan

a. Perbandingan Algoritma Digital Signature

Pada sistem digital signature, algoritma utama yang digunakan adalah RSA 2048-bit. Sebelum memilih RSA, alternatif lain yang dipertimbangkan adalah ECC (Elliptic Curve Cryptography).

Algoritma	Kelebihan	Kekurangan
RSA 2048-bit	Stabil, umum digunakan, dokumentasi lengkap, implementasi mudah	Proses signing dan verifikasi lebih lambat dibanding ECC
ECC	Ukuran key lebih kecil, performa lebih cepat	Implementasi lebih kompleks

RSA dipilih karena lebih umum digunakan pada sistem digital signature dan memiliki dokumentasi serta library yang lebih lengkap pada Python. Selain itu, implementasi RSA pada framework Flask lebih mudah diterapkan dan sesuai dengan kebutuhan aplikasi. Penggunaan RSA 2048-bit juga masih dianggap aman untuk implementasi digital signature modern.

b. Algoritma Hashing Dokumen

Untuk menjaga integritas dokumen, sistem menggunakan SHA-256 sebagai algoritma hashing. Sebelum memilih SHA-256, algoritma MD5 dan SHA-1 juga dipertimbangkan.

Algoritma	Kelebihan	Kekurangan
-----------	-----------	------------

SHA-256	Aman, collision resistant, masih banyak digunakan	Proses sedikit lebih berat
MD5	Proses hashing cepat	Rentan collision dan tidak aman
SHA-1	Lebih baik dari MD5	Sudah memiliki kelemahan keamanan

SHA-256 dipilih karena memiliki tingkat keamanan yang lebih baik dibanding MD5 dan SHA-1. Pada aplikasi SignDoc, algoritma ini digunakan untuk menghasilkan hash dokumen sebelum proses digital signature dilakukan. Hash dokumen akan berubah apabila isi dokumen mengalami perubahan sehingga sistem dapat mendeteksi modifikasi dokumen saat proses verifikasi. Selain itu, SHA-256 masih banyak digunakan pada berbagai sistem keamanan digital modern karena sulit dimanipulasi dan memiliki tingkat keamanan yang baik.

c. Algoritma Password Hashing

Pada proses autentikasi pengguna, sistem menggunakan bcrypt untuk hashing password sebelum disimpan ke database. Alternatif lain yang dipertimbangkan adalah MD5 dan SHA-1 hashing untuk password.

Algoritma	Kelebihan	Kekurangan
bcrypt	Lebih aman dan tahan brute force	Proses hashing lebih lambat
MD5 Password Hashing	Cepat dan ringan	Mudah dicrack
SHA-1 Password Hashing	Lebih baik dari MD5	Sudah kurang aman

bcrypt dipilih karena memiliki tingkat keamanan yang lebih baik untuk penyimpanan password pengguna. bcrypt dirancang dengan proses hashing yang lebih lambat sehingga lebih tahan terhadap serangan brute force dibanding hashing biasa. Dengan penggunaan bcrypt, password pengguna tidak disimpan dalam bentuk asli sehingga keamanan data pengguna menjadi lebih terjaga.

d. Database

Pada aplikasi SignDoc, database yang digunakan adalah MySQL untuk menyimpan data pengguna, dokumen, dan digital signature. Sebelum menggunakan MySQL, SQLite juga dipertimbangkan sebagai alternatif database pada sistem.

Database	Kelebihan	Kekurangan
MySQL	Stabil, mendukung multi-user, mudah diintegrasikan	Membutuhkan konfigurasi server
SQLite	Ringan dan sederhana	Kurang cocok untuk aplikasi multi user

MySQL dipilih karena lebih sesuai untuk aplikasi web berbasis multi-user seperti SignDoc. Selain mampu menangani banyak data dan koneksi pengguna, MySQL juga lebih mudah diintegrasikan dengan Flask dan mendukung pengelolaan relasi data pada sistem.

D. Implementasi

1. Potongan Kode Kritis

a. Generate RSA Keypair

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
    backend=default_backend()
)
```

Potongan kode di atas digunakan untuk menghasilkan pasangan public key dan private key menggunakan algoritma RSA 2048-bit. Pada aplikasi SignDoc, proses generate keypair dilakukan secara otomatis saat pengguna melakukan registrasi akun. Public key digunakan untuk proses verifikasi signature, sedangkan private key digunakan untuk proses digital signature dokumen.

b. Enkripsi RSA Keypair

```
encrypted_private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.BestAvailableEncryption(
password.encode())
).decode('utf-8')
```

Potongan kode tersebut digunakan untuk mengenkripsi private key sebelum disimpan ke database pada aplikasi SignDoc. Proses enkripsi menggunakan password pengguna sehingga private key tidak dapat diakses atau digunakan secara langsung tanpa proses autentikasi pengguna terlebih dahulu. Dengan cara ini, keamanan private key dapat lebih terjaga apabila terjadi akses tidak sah pada sistem atau database.

c. Deskripsi Private Key

```
return serialization.load_pem_private_key(
    encrypted_pem.encode(),
    password=password.encode(),
    backend=default_backend()
)
```

Potongan kode ini digunakan untuk memuat dan mendekripsi private key saat proses digital signature dilakukan pada aplikasi SignDoc. Private key yang tersimpan dalam bentuk terenkripsi hanya dapat digunakan apabila password pengguna yang dimasukkan sesuai. Dengan mekanisme ini, sistem dapat memastikan bahwa private key hanya dapat diakses oleh pemilik akun yang sah.

d. Hashing Dokumen

```
def hash_file(filepath: str) -> str:
    """
    sha256 = hashlib.sha256()
    with open(filepath, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            sha256.update(chunk)
```



```
return sha256.hexdigest()
```

Potongan kode di atas digunakan untuk menghitung hash dokumen menggunakan algoritma SHA-256 pada aplikasi SignDoc. Proses hashing dilakukan dengan membaca file per bagian (chunk) agar tetap efisien ketika memproses dokumen berukuran besar. Hash dokumen tersebut digunakan sebelum proses digital signature dilakukan untuk menjaga integritas dokumen serta mendeteksi perubahan pada isi file.

e. Digital Signature

```
def sign_document(private_key, file_hash_hex: str) -> str:

    hash_bytes = bytes.fromhex(file_hash_hex)

    signature = private_key.sign(
        hash_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

Potongan kode di atas digunakan untuk melakukan proses digital signature terhadap hash dokumen pada aplikasi SignDoc. Hash dokumen terlebih dahulu dikonversi dari bentuk hexadecimal menjadi bytes sebelum proses signing dilakukan. Selanjutnya, sistem menggunakan private key RSA pengguna dengan metode RSA-PSS padding dan algoritma SHA-256 untuk menghasilkan signature dokumen yang aman.

f. Verifikasi Signature

```
public_key.verify(
    signature,
    hash_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

Potongan kode di atas digunakan untuk melakukan proses verifikasi digital signature pada aplikasi SignDoc. Sistem menggunakan public key milik penanda tangan untuk memverifikasi kecocokan antara signature dan hash dokumen. Proses verifikasi menggunakan RSA-PSS padding dan algoritma SHA-256 agar keamanan signature tetap terjaga. Jika proses verifikasi berhasil, maka dokumen dinyatakan valid dan tidak mengalami perubahan setelah proses penandatanganan dilakukan.

2. Penjelasan Keputusan Implementasi

Pada aplikasi SignDoc, sistem digital signature diimplementasikan menggunakan algoritma RSA 2048-bit karena memiliki tingkat keamanan yang baik

dan masih umum digunakan dalam sistem keamanan digital modern. RSA digunakan dalam proses penandatanganan (signing) dan verifikasi dokumen melalui mekanisme pasangan kunci public key dan private key.

Untuk menjaga integritas dokumen, sistem menggunakan SHA-256 sebagai fungsi hashing yang dilakukan sebelum proses signing. Proses hashing juga dilakukan secara bertahap (chunk) agar sistem tetap efisien dalam menangani dokumen berukuran besar. Hal ini diperlukan karena RSA tidak memproses data berukuran besar secara langsung, sehingga dokumen harus direpresentasikan dalam bentuk hash terlebih dahulu.

Pada proses penandatanganan dan verifikasi, sistem menggunakan RSA-PSS (Probabilistic Signature Scheme) sebagai metode padding. RSA-PSS dipilih karena memiliki tingkat keamanan yang lebih tinggi dibandingkan padding lama seperti PKCS#1 v1.5, serta bersifat probabilistik sehingga setiap proses penandatanganan dapat menghasilkan signature yang berbeda meskipun dokumen yang digunakan sama.

Selain itu, sistem dibangun menggunakan framework Flask karena bersifat ringan, fleksibel, dan mudah diintegrasikan dengan library kriptografi. Untuk meningkatkan keamanan, private key pengguna dienkripsi menggunakan password sebelum disimpan ke dalam sistem. Sistem juga menyediakan fitur QR Code untuk mempermudah proses verifikasi dokumen, serta mendukung multi-signer yang memungkinkan penandatanganan dokumen oleh lebih dari satu pihak.

E. Pengujian

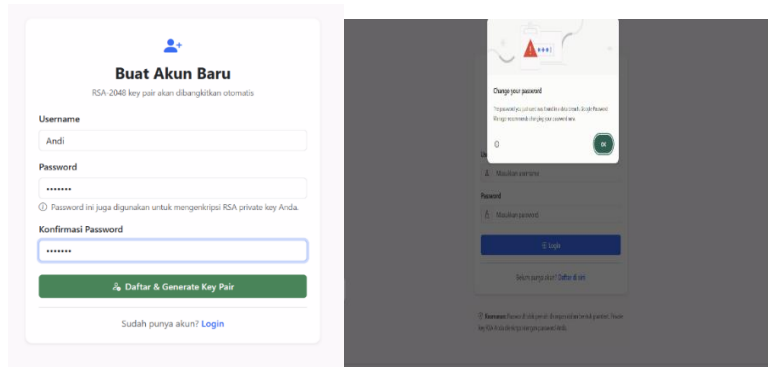
1. Skenario Normal

a. Registrasi Pengguna

Komponen	Keterangan
Tujuan Pengujian	Memastikan pengguna dapat membuat akun baru pada sistem
Langkah Pengujian	Pengguna mengisi username, dan password kemudian menekan tombol daftar
Hasil yang diharapkan	Akun berhasil dan data pengguna tersimpan ke database
Hasil Pengujian	Berhasil

Pada pengujian ini, sistem berhasil melakukan proses registrasi pengguna dan secara otomatis membuat RSA keypair untuk pengguna baru. Public key dan private key terenkripsi berhasil disimpan ke database.

Screenshot Pengujian:

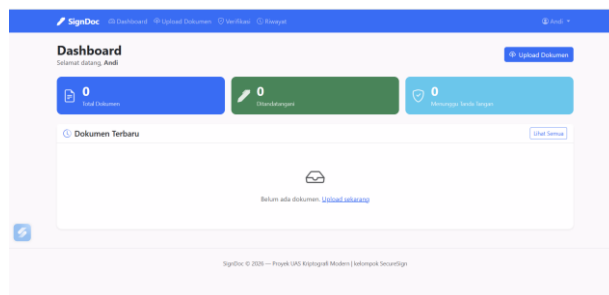


b. Login Pengguna

Komponen	Keterangan
Tujuan Pengujian	Memastikan pengguna dapat login ke sistem
Langkah Pengujian	Pengguna memasukkan username dan password yang benar
Hasil yang diharapkan	Sistem menampilkan dash
Hasil Pengujian	Berhasil

Sistem berhasil melakukan autentikasi pengguna menggunakan password yang telah di-hash menggunakan bcrypt. Setelah login berhasil, pengguna dapat mengakses fitur upload, signing, dan verifikasi dokumen.

Screenshot Pengujian:



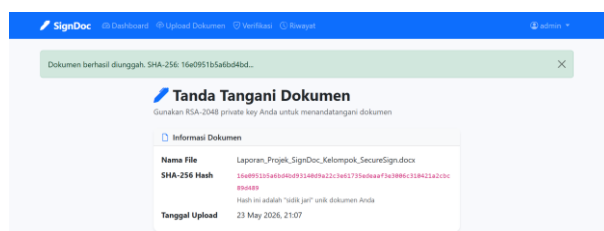
c. Upload dokumen

Komponen	Keterangan
Tujuan Pengujian	Memastikan pengguna dapat mengupload dokumen ke sistem
Langkah Pengujian	Pengguna memilih file dokumen kemudian menekan tombol upload

Hasil yang diharapkan	Dokumen berhasil tersimpan pada folder uploads dan data dokumen masuk ke database
Hasil Pengujian	Berhasil

Sistem berhasil menerima file dokumen yang diupload pengguna. Dokumen tersimpan pada folder uploads dan informasi dokumen berhasil dicatat ke database. Sistem juga berhasil membaca file untuk proses hashing sebelum dilakukan digital signature.

Screenshot Pengujian:

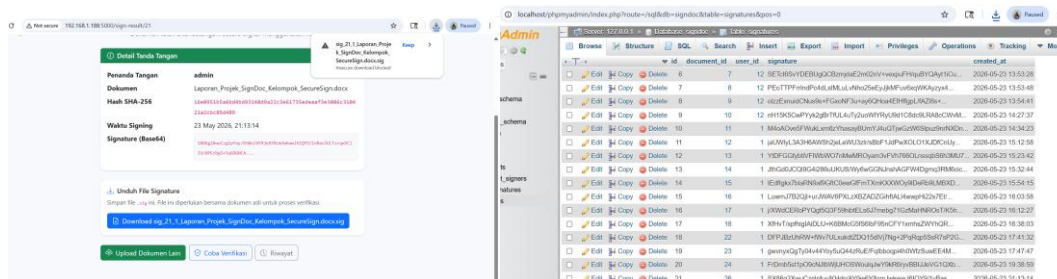


d. Digital Signature Dokumen

Komponen	Keterangan
Tujuan Pengujian	sistem dapat melakukan proses digital signature pada dokumen
Langkah Pengujian	Pengguna memilih dokumen kemudian menekan tombol sign
Hasil yang diharapkan	Sistem menghasilkan digital signature dan menyimpannya ke database
Hasil Pengujian	Berhasil

Sistem berhasil membuat hash dokumen menggunakan algoritma SHA-256 kemudian melakukan proses signing menggunakan private key RSA pengguna. Signature berhasil dibuat dan disimpan ke database serta folder signatures. Sistem juga menampilkan status bahwa proses penandatanganan dokumen berhasil dilakukan.

Screenshot Pengujian:

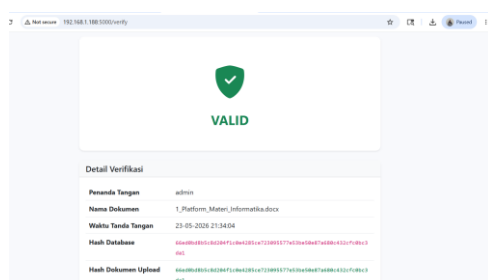


e. Verifikasi Dokumen Valid

Komponen	Keterangan
Tujuan Pengujian	Memastikan sistem dapat memverifikasi dokumen yang asli dan belum dimodifikasi
Langkah Pengujian	Pengguna mengupload dokumen asli beserta file signature
Hasil yang diharapkan	Sistem menampilkan status dokumen valid
Hasil Pengujian	Berhasil

Sistem berhasil menghitung ulang hash dokumen dan mencocokkannya dengan signature menggunakan public key pengguna. Karena dokumen tidak mengalami perubahan, proses verifikasi berhasil dan sistem menampilkan status bahwa dokumen valid serta signature sesuai

Screenshot Pengujian:



2. Skenario error

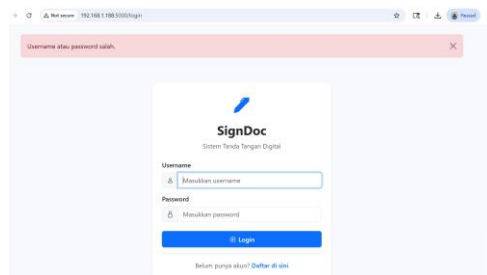
a. Login dengan Password Salah

Komponen	Keterangan
Tujuan Pengujian	Memastikan sistem menolak login dengan password yang salah

Langkah Pengujian	Pengguna memasukkan username benar tetapi password salah
Hasil yang diharapkan	Sistem menampilkan pesan login gagal
Hasil Pengujian	Berhasil

Sistem berhasil mendeteksi password yang tidak sesuai dengan hash bcrypt pada database. Pengguna tidak dapat masuk ke dashboard dan sistem menampilkan pesan kesalahan login.

Screenshot Pengujian:

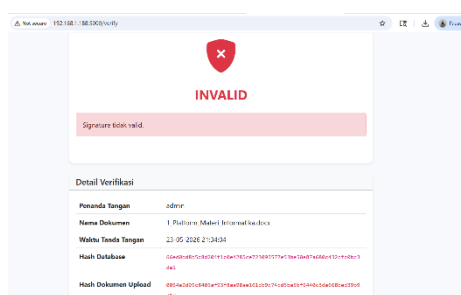


3. Skenario Serangan

a. Verifikasi Dokumen yang Dimodifikasi dan Verifikasi Menggunakan Signature yang Tidak Sesuai

Komponen	Keterangan
Tujuan Pengujian	sistem menolak signature yang bukan milik dokumen , dan sistem dapat mendeteksi perubahan pada dokumen
Langkah Pengujian	Dokumen yang sudah ditandatangani diubah isinya kemudian diverifikasi Kembali, dan Pengguna mengupload dokumen dengan file signature milik dokumen lain
Hasil yang diharapkan	Sistem menampilkan status dokumen tidak valid dan signature tidak valid
Hasil Pengujian	Berhasil

Screenshot Pengujian:



SignDoc berhasil menjalankan fitur utama sistem digital signature dengan baik. Sistem mampu melakukan registrasi pengguna, login, upload dokumen, proses digital signature, dan verifikasi dokumen menggunakan algoritma RSA 2048-bit dan SHA-256. Selain itu, sistem juga berhasil mendeteksi kesalahan autentikasi, file tidak valid, serta perubahan isi dokumen setelah proses penandatanganan dilakukan. Dengan demikian, implementasi kriptografi pada aplikasi berhasil menjaga keaslian, integritas, dan keamanan dokumen digital

F. Keterbatasan dan Refleksi

a. Keterbatasan Sitem

1. Sistem masih menggunakan session password sementara pada proses penandatanganan.
2. Sistem belum sepenuhnya menggunakan protokol HTTPS secara penuh.
3. Sistem belum dilengkapi dengan fitur Certificate Authority (CA).
4. Sistem belum mendukung penggunaan digital timestamp yang bersifat resmi.
5. Sistem belum memiliki mekanisme key revocation (pencabutan kunci).
6. Autentikasi pengguna belum menggunakan mekanisme multi-faktor (MFA).
7. Verifikasi melalui QR Code belum bersifat real-time, karena sistem belum melakukan pengecekan ulang secara otomatis terhadap perubahan dokumen saat proses pemindaian. Akibatnya, dokumen yang telah dimodifikasi masih dapat terdeteksi sebagai valid dalam kondisi tertentu.
8. Sistem belum dioptimalkan untuk penggunaan dalam skala besar (high traffic).

b. Refleksi

Berdasarkan proses implementasi yang telah dilakukan, sistem SignDoc berhasil mengembangkan fitur utama berupa digital signature menggunakan algoritma RSA dan SHA-256. Sistem ini juga telah mendukung fitur multi-signer serta mekanisme verifikasi dokumen melalui QR Code untuk mempermudah pengguna dalam melakukan validasi dokumen digital.

Selama proses pengembangan, dapat dipahami bahwa penerapan kriptografi pada sistem web membutuhkan perhatian khusus, terutama pada aspek keamanan dan manajemen kunci. Meskipun sistem telah menggunakan enkripsi pada private key serta algoritma kriptografi yang cukup kuat, masih terdapat ruang pengembangan untuk meningkatkan keamanan dan keandalan sistem secara keseluruhan.

Apabila terdapat waktu pengembangan lebih lanjut, sistem ini dapat ditingkatkan dengan penambahan fitur seperti Certificate Authority, digital timestamp, key revocation, serta mekanisme verifikasi real-time pada QR Code. Selain itu, sistem juga dapat dikembangkan dengan peningkatan keamanan komunikasi menggunakan HTTPS secara penuh serta penerapan autentikasi multi-faktor agar sistem menjadi lebih aman dan sesuai dengan standar keamanan modern.